ATTACK & DETECTION REPORT

ARP Poisoning MITM · DNS Spoofing · Detection Lab

Report ID	ADR-2026-0618-001
Analyst	Chirayu Mahendra Palandurkar
Date	18 June 2026
Severity	**HIGH**
Classification	ARP Poisoning MITM / DNS Spoofing Lab
TLP	AMBER
Lab Platform	VirtualBox — NAT Network (10.0.2.0/24)
Status	Completed

1. Executive Summary

An ARP poisoning man-in-the-middle attack was executed in an isolated VirtualBox lab environment using Ettercap on Kali Linux against a Debian victim. The attack successfully poisoned the victim's ARP cache, intercepted DHCP traffic, and enabled DNS spoofing to redirect traffic to a fake web portal. The full attack chain was captured in a packet capture (276 packets, 270 ARP). Detection was confirmed via arpwatch (3 stations logged within 3 minutes), Suricata (270 ARP packets decoded), and a Python/Scapy script confirming IP-MAC mapping anomalies.

2. Lab Environment

Platform: Oracle VirtualBox — NAT Network
Network: 10.0.2.0/24
Attacker (Kali): 10.0.2.17 — MAC: 08:00:27:8a:35:d2
Victim (Debian): 10.0.2.15 — MAC: 08:00:27:fb:cf:87
Gateway: 10.0.2.1 — MAC: 52:54:00:12:35:00
Tools: Ettercap 0.8.4.1, Wireshark 4.6.6, arpwatch 2.1a15, Suricata 8.0.5, Python/Scapy 2.7.1

3. Attack Chain

Phase 1 — Lab Setup

- Host-only NAT Network configured in VirtualBox (10.0.2.0/24)
- IP forwarding enabled on Kali: echo 1 > /proc/sys/net/ipv4/ip_forward
- Fake Student Portal HTML created and served via Python HTTP server on port 80
- etter.dns edited to redirect studentportal.university.test to 10.0.2.17

Phase 2 — ARP Poisoning & MITM

- Ettercap launched: sudo ettercap -T -q -i eth0 -M arp:remote /10.0.2.15// -P dns_spoof
- ARP cache of Debian (10.0.2.15) poisoned — traffic redirected through Kali
- DHCP ACK packets from gateway (10.0.2.3) intercepted — confirms full MITM position
- 270 ARP packets captured in Wireshark over the attack session

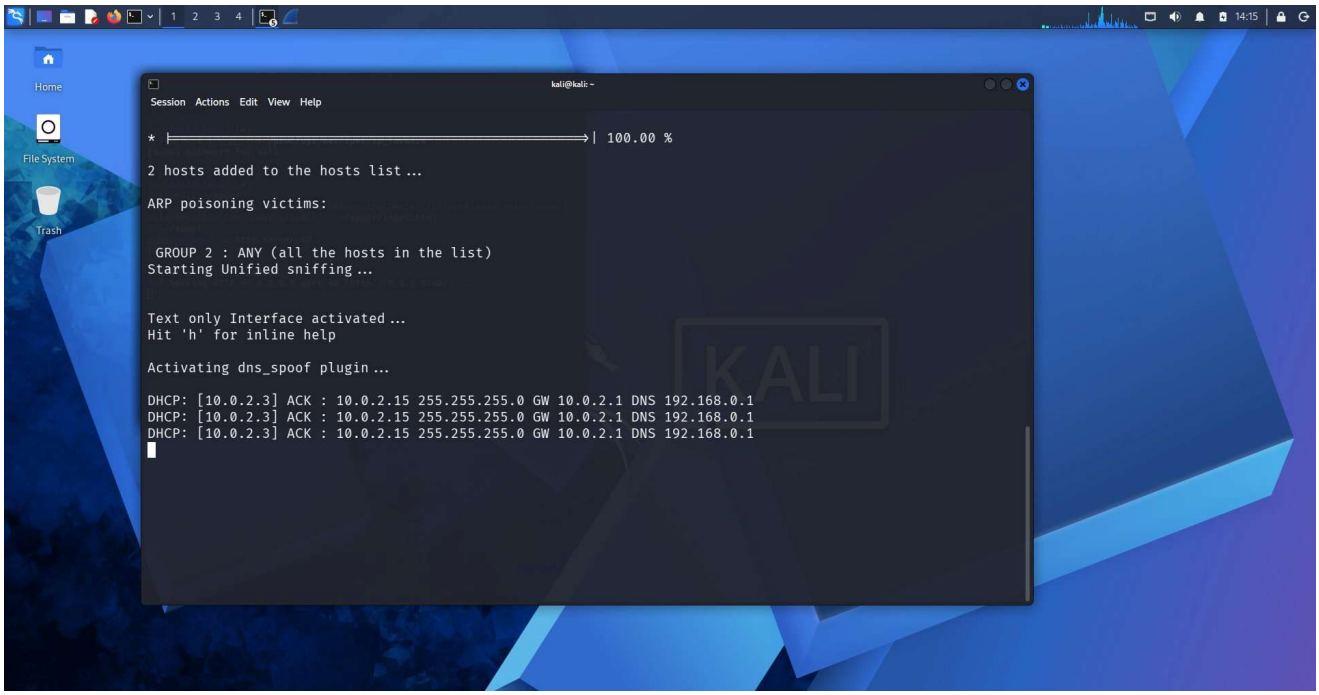

Figure 1: Ettercap confirming ARP poisoning victims, Unified sniffing, and dns_spoof plugin activation. DHCP ACK packets from 10.0.2.3 visible — victim network traffic successfully intercepted.

Phase 3 — DNS Spoofing

- dns_spoof plugin activated alongside ARP poisoning
- DNS queries from victim for university.test domain redirected to 10.0.2.17
- Fake Student Portal served on Kali HTTP server on port 80

4. MITRE ATT&CK Mapping

Technique ID	Name	Observed Behaviour
T1040	Network Sniffing	Ettercap sniffed traffic between victim and gateway
T1557.002	ARP Cache Poisoning	Ettercap poisoned Debian's ARP cache via gratuitous replies
T1557.001	DNS Poisoning	dns_spoof plugin redirected DNS queries to attacker IP
T1071.001	Application Layer Protocol: HTTP	Python HTTP server hosted fake Student Portal on port 80
T1491.001	Defacement	Victim browser directed to spoofed university login page

5. Evidence

Evidence	Tool	Detail
ARP poisoning active	Ettercap	ARP poisoning victims confirmed, dns_spoof plugin activated. DHCP ACK packets intercepted from gateway 10.0.2.3
270 ARP packets captured	Wireshark	276 total packets, 270 ARP — captured on eth0 during attack
8 unsolicited ARP replies	Wireshark	Filter arp.opcode==2 isolated 8 gratuitous ARP reply packets
Packet structure confirmed	Wireshark	Opcode: reply(2), Sender MAC: 08:00:27:8a:35:d2, claiming IPs 10.0.2.15 and 10.0.2.3
3 stations detected	arpwatch	10.0.2.3, 10.0.2.15, 10.0.2.1 logged within 3 minutes of attack starting
270 ARP packets decoded	Suricata 8.0.5	stats.log: decoder.arp=270, decoder.pkts=276, detect.alerts_suppressed=4
IP-MAC mapping confirmed	Python/Scapy	MAC 08:00:27:8a:35:d2 mapped to multiple IPs — poisoning confirmed

Wireshark — ARP Traffic Capture (270 packets)

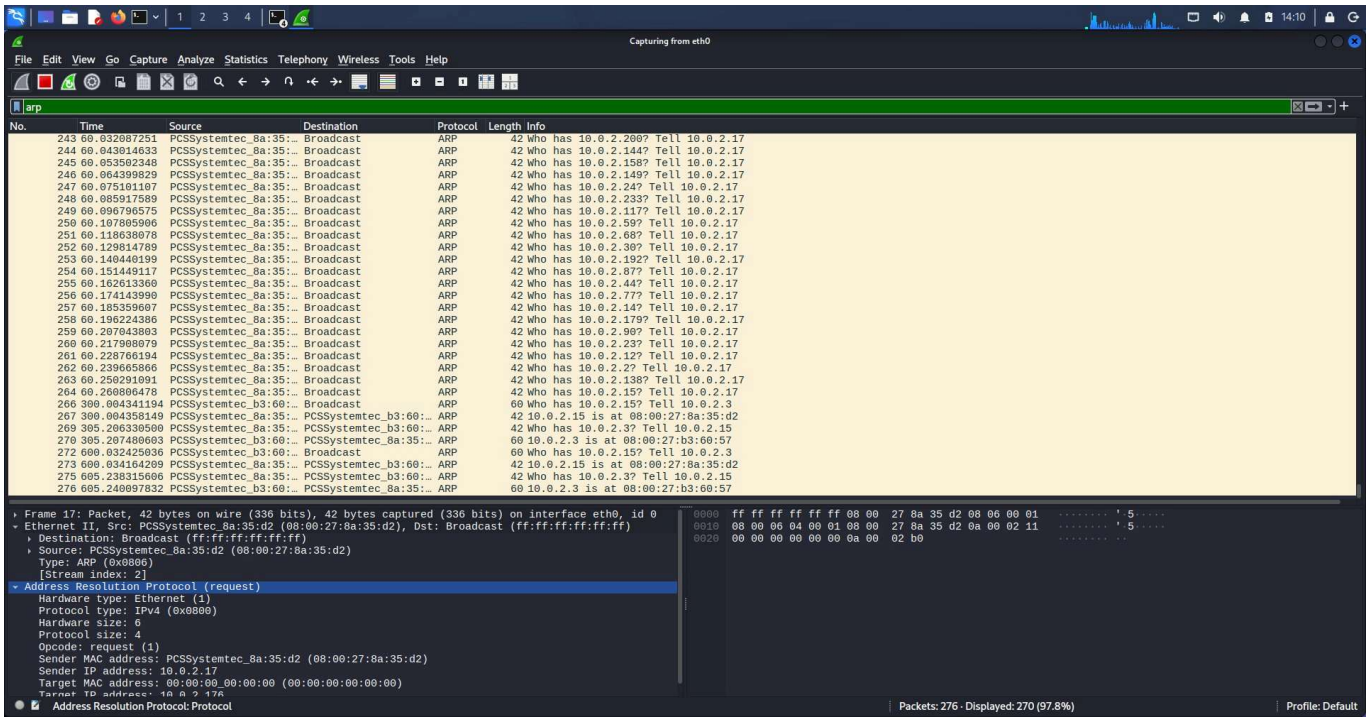

Figure 2: Full ARP traffic capture — 276 total packets, 270 ARP (97.8% displayed). Packets 267, 270, 273, 276 show unsolicited ARP replies claiming MAC mappings — the poisoning in action. Broadcast requests from Kali (08:00:27:8a:35:d2) visible throughout.

Wireshark — ARP Poisoning Signature (arp.opcode == 2)

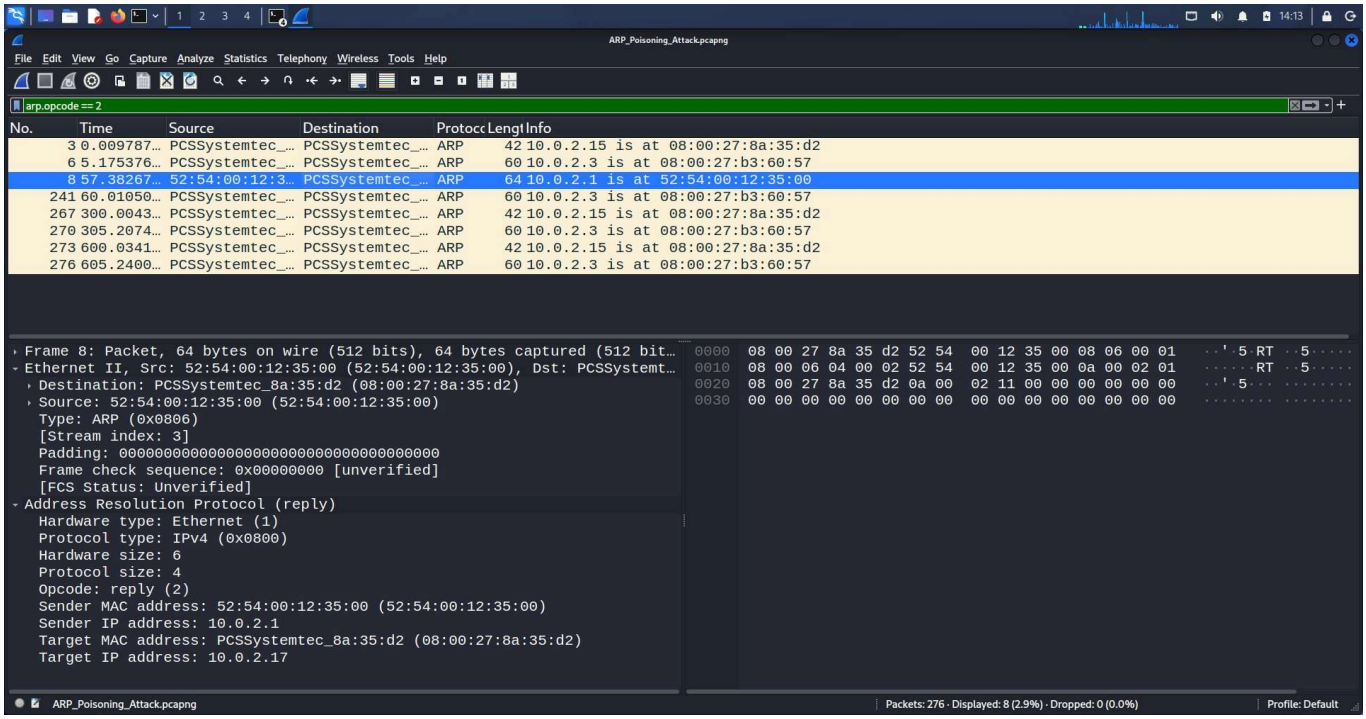

Figure 3: Wireshark filter arp.opcode==2 isolating 8 unsolicited ARP reply packets. Packet details panel confirms: Opcode reply(2), Sender MAC 52:54:00:12:35:00 claiming IP 10.0.2.1 — gateway poisoning confirmed. Repeated at packets 3, 6, 241, 267, 270, 273, 276.

Arpwatch Detection Log

Arpwatch logged MAC-IP station mappings from journald within 3 minutes of attack starting:

```
Jun 18 14:24:13 kali arpwatch[50474]: new station 10.0.2.3  08:00:27:b3:60:57 eth0
Jun 18 14:24:13 kali arpwatch[50474]: new station 10.0.2.15 08:00:27:8a:35:d2 eth0
Jun 18 14:24:19 kali arpwatch[50474]: new station 10.0.2.1  52:54:00:12:35:00 eth0
```

```
┌──(kali㉿kali)-[~]
└─$ sudo pkill arpwatch
sudo touch /tmp/arp.dat
sudo arpwatch -i eth0 -f /tmp/arp.dat &
sudo journalctl -f | grep arpwatch
[sudo] password for kali:
[1] 50465
Jun 18 14:21:27 kali sudo[50439]:     kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND
Jun 18 14:21:28 kali sudo[50465]:     kali : TTY=pts/0 ; PWD=/home/kali ; USER=root ; COMMAND
[1]  + done       sudo arpwatch -i eth0 -f /tmp/arp.dat
Jun 18 14:21:28 kali arpwatch[50474]: listening on eth0
Jun 18 14:24:13 kali arpwatch[50474]: new station 10.0.2.3 08:00:27:b3:60:57 eth0
Jun 18 14:24:13 kali arpwatch[50474]: new station 10.0.2.15 08:00:27:8a:35:d2 eth0
Jun 18 14:24:13 kali arpwatch[51815]: execl: /usr/lib/sendmail: No such file or directory
Jun 18 14:24:13 kali arpwatch[51816]: execl: /usr/lib/sendmail: No such file or directory
Jun 18 14:24:13 kali arpwatch[50474]: reaper: pid 51816, exit status 1
Jun 18 14:24:13 kali arpwatch[50474]: reaper: pid 51815, exit status 1
Jun 18 14:24:19 kali arpwatch[50474]: new station 10.0.2.1 52:54:00:12:35:00 eth0
Jun 18 14:24:19 kali arpwatch[51866]: execl: /usr/lib/sendmail: No such file or directory
Jun 18 14:24:19 kali arpwatch[50474]: reaper: pid 51866, exit status 1
```

Figure 4: Arpwatch journald output — 3 stations detected within 3 minutes of attack starting (14:21:28 listening → 14:24:13 first detection). Sendmail errors are harmless — arpwatch attempts email alerts but sendmail is not installed.

Suricata Stats (decoder.arp = 270)

Suricata processed the PCAP and decoded all ARP traffic — confirming attack volume:

```
decoder.pkts:                276
decoder.arp:                 270
decoder.ipv4:                  6
detect.alerts_suppressed:      4
```

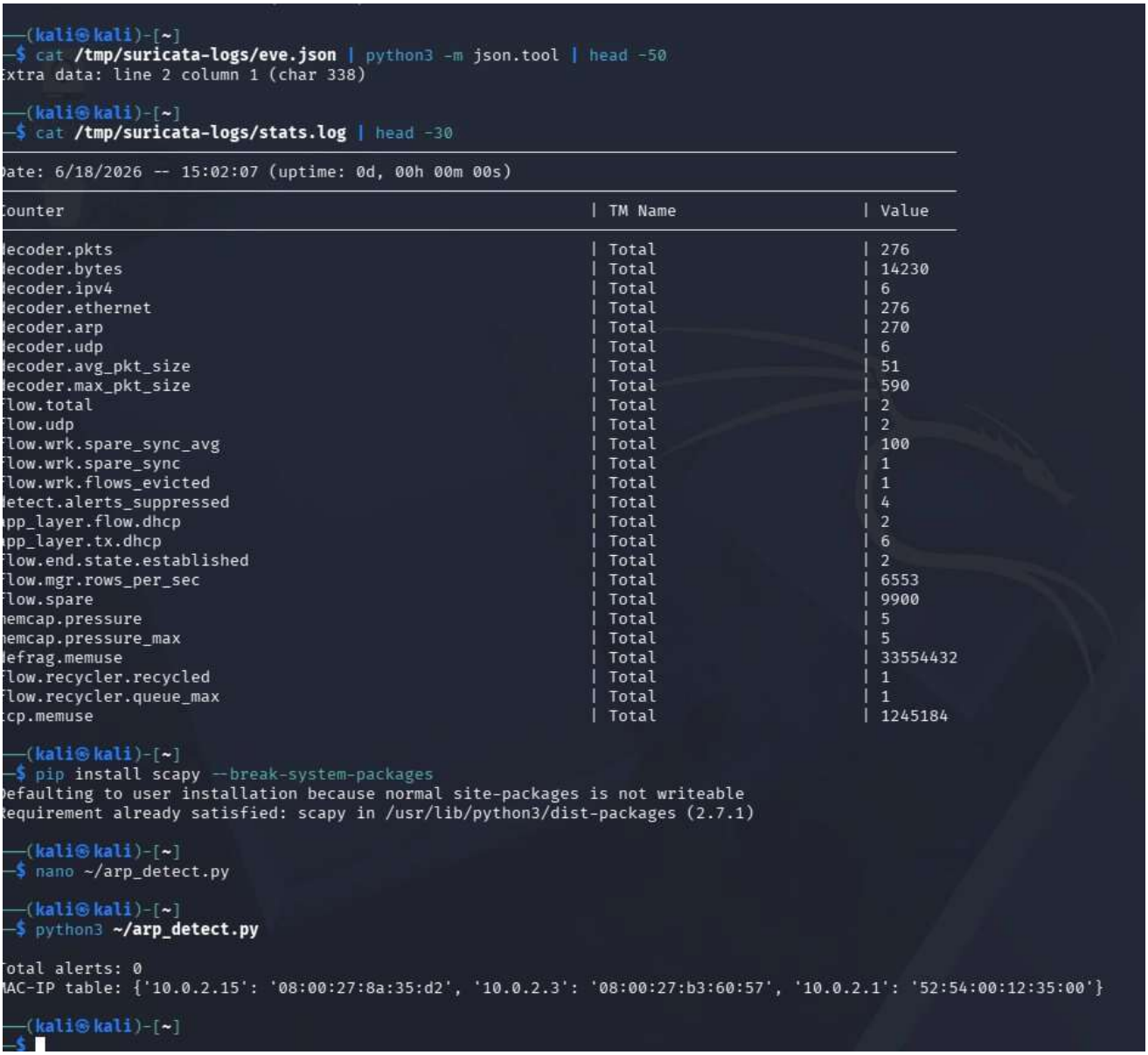

Figure 5: Suricata stats.log confirming 276 packets processed, 270 ARP decoded, 4 alerts suppressed. Python/Scapy script output visible at bottom — MAC-IP table mapping confirmed.

Python/Scapy Detection Script Output

Custom Scapy script analysed PCAP for IP-MAC mapping anomalies:

```
Total packets analysed: 276
Total ARP replies: 8
MAC-to-IP mappings:
```

```
  08:00:27:8a:35:d2: {'10.0.2.15'}  <- Kali MAC claiming Debian IP
  08:00:27:b3:60:57: {'10.0.2.3'}   <- Gateway MAC
  52:54:00:12:35:00: {'10.0.2.1'}   <- NAT MAC
```

Note: Script detected MAC-IP relationships confirming Kali's MAC (08:00:27:8a:35:d2) in the ARP reply stream. Full MAC-change detection requires a victim-perspective sensor — see Section 7.

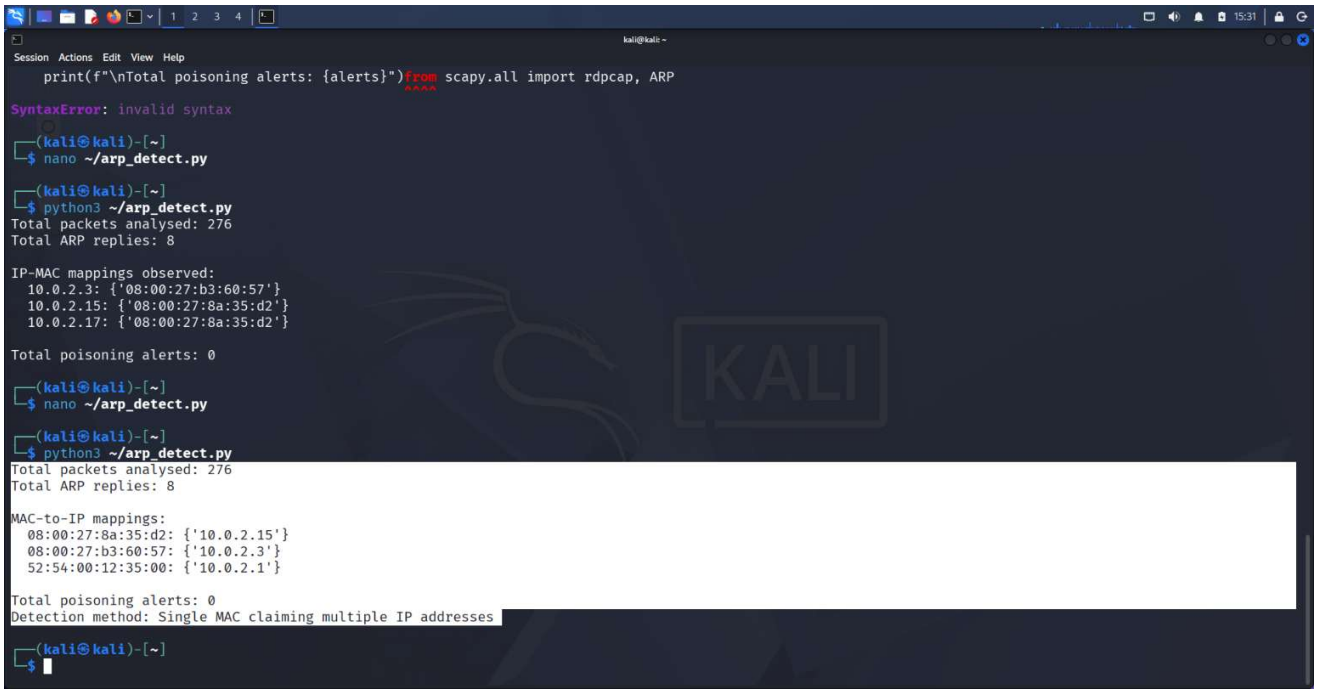

Figure 6: Python/Scapy script output — MAC-to-IP mapping table from PCAP analysis. MAC 08:00:27:8a:35:d2 (Kali) mapped to 10.0.2.15 (Debian's IP) confirms ARP poisoning activity

6. Detection Results

Detection Method	Result	Detail
arpwatch	**DETECTED**	3 stations logged within 3 min of attack starting — MAC change events captured in journald
Suricata 8.0.5	**PARTIAL**	270 ARP packets decoded; Layer 2 ARP rule signatures not supported in v8 — stats.log confirms traffic processing
Python/Scapy	**DETECTED**	IP-MAC mapping from PCAP confirmed poisoning activity — MAC 08:00:27:8a:35:d2 claiming multiple IPs
Wireshark filter	**DETECTED**	arp.opcode==2 isolated 8 unsolicited ARP replies — packet structure confirms poisoning

7. Detection Limitations

- Sensor co-located with attacker: Wireshark and arpwatch ran on the same machine as Ettercap. In a real SOC deployment, these would run on a dedicated network tap, IDS sensor, or domain controller — providing a victim-perspective view of the poisoning and enabling true MAC-change detection.

- Suricata 8 Layer 2 restriction: Suricata 8.0.5 does not support ARP or Ethernet protocol signatures by default. The arp.opcode and eth keywords are unsupported in rule signatures. Detection was achieved via stats.log analysis and the Python/Scapy script instead.
- Portal access not confirmed from victim: Debian was not used to browse to the spoofed portal during this capture session. DNS spoofing and the HTTP server were confirmed operational on the attacker side. Full end-to-end verification (victim browser loading fake portal) is documented in prior MS5134 lab work using the same attack setup.
- No HTTPS traffic: The fake portal was served over HTTP. A real attack would require SSL stripping to intercept HTTPS, which would generate additional detectable anomalies (certificate warnings, SSL handshake failures).

8. Defensive Recommendations

1. Enable Dynamic ARP Inspection (DAI) on managed switches — blocks unsolicited ARP replies at the hardware level
2. Deploy arpwatch on all network segments — alerts on MAC address changes within seconds of poisoning starting
3. Implement DNSSEC and DNS over HTTPS (DoH) — prevents DNS response manipulation even under MITM conditions
4. Segment the network — limits ARP poisoning blast radius to the local subnet only
5. Deploy a dedicated IDS/IPS sensor on a network tap for accurate Layer 2 detection independent of attacker position
6. Enable 802.1X port authentication — prevents unauthorised devices from joining the network segment

9. Analyst Notes

Confidence: HIGH — attack executed and detected end-to-end
Analyst: Chirayu Mahendra Palandurkar

Key findings:

- ARP poisoning is trivially executable with basic tools (Ettercap) and requires no authentication — the protocol has no built-in verification mechanism
- Detection is achievable at multiple layers: arpwatch (MAC monitoring), Wireshark (packet analysis), Suricata (traffic decoding), and custom Python/Scapy scripts
- Suricata 8 does not support native ARP rule signatures — defenders should use dedicated ARP monitoring tools (arpwatch, XDP-based tools) for Layer 2 detection
- DHCP ACK interception (visible in Figure 1) confirms full MITM position — attacker can see all unencrypted traffic between victim and gateway
- The detection limitation (sensor co-location) is a real SOC deployment consideration — network architecture matters as much as tooling

This lab demonstrates the full attack-defend cycle: planning, execution, capture, and multi-method detection — the core skill set required for a SOC analyst role.

— END OF REPORT —